SkillPulse Cluster Deployment Guide

Complete Local Kubernetes Setup, Configuration Fixes, and Architecture Mapping

1. Overview & Setup Prerequisites

This document acts as an official deployment runbook for hosting the multi-tier **SkillPulse Application Stack** (comprising Frontend, Backend, and MySQL Database Layers) on a localized infrastructure utilizing Kubernetes container orchestration primitives via **Kind (Kubernetes in Docker)**.

ENVIRONMENT REQUIREMENTS

Ensure the following base system platforms are initialized and responsive:

- **Docker Desktop:** Must be executing in the host environment background.
- **WSL 2 Core Engine:** Ubuntu Linux distribution terminal profile context.
- **CLIs:** Validated binary paths for `kubectl` and `kind`.

2. Cluster Initialization

In scenarios where the local deployment target node is non-responsive, disabled, or missing from the operational thread context, initialize the specific cluster architecture context by running the following structural blocks:

```
# Initialize a new isolated Kind cluster node matching the system context schema
kind create cluster --name skillpulse

# Validate that the active context pointer successfully targets the configuration
kubectl config current-context
# Expected Verification Return: kind-skillpulse
```

3. Offline Image Loading (Kind Isolation Bypass)

Because the local Kind cluster orchestrates operations inside isolated Docker nodes, it may encounter network mapping failures (such as `ErrImagePull` or `ImagePullBackOff`) when pulling containers dynamically from external registries. Inject local runtime binaries directly using:

```
# Intercept the host container stack and copy the binary directly to the Kind node
kind load docker-image mysql:8.0 --name skillpulse
```

4. Verified Manifest Configurations

To avoid structural parsing issues such as `yaml: line 18: could not find expected ':'`, the corrected structured deployment schema for `20-backend.yaml` has been unified. This file integrates both the networking Service (NodePort) and the operational Deployment workload.

```
apiVersion: v1
kind: Service
metadata:
  name: backend
  namespace: skillpulse
  labels:
    app: backend
spec:
  type: NodePort
  selector:
    app: backend
  ports:
  - name: http
    port: 8080
    targetPort: 8080
    nodePort: 30080
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend
  namespace: skillpulse
spec:
  replicas: 1
  selector:
    matchLabels:
      app: backend
  template:
    metadata:
      labels:
        app: backend
    spec:
      containers:
      - name: backend
        image: trainwithshubham/skillpulse-backend:latest
        imagePullPolicy: IfNotPresent
        ports:
        - name: http
          containerPort: 8080
```

To safely rewrite the corrupted runtime block automatically on the server disk, use this terminal echo macro command directly inside the `k8s/` directory:

```
rm -f 20-backend.yaml
cat << 'EOF' > 20-backend.yaml
[Paste the YAML block structure here]
EOF
kubectl apply -f 20-backend.yaml
```

5. Cluster Deployment Execution

Workloads should be initialized from within the configuration directory. Numbered sequences verify proper dependency flow during application bootstrap:

```
# Navigate to the Kubernetes context tracking directory
cd k8s/

# Deploy full localized environment nodes structurally
kubectl apply -f .
```

Verifying Component Readiness State

```
# Monitor live container allocation inside the targeted namespace environment
kubectl get pods -n skillpulse -w
```

6. Localhost Networking & Access Architecture

Option A: Dynamic Port-Forwarding

Temporary session proxy tunnel mapping to route local application ports through specific shell listeners. Requires continuous terminal availability.

To port-forward the API Backend:

```
kubectl port-forward svc/backend
8080:8080 -n skillpulse --address 0.0.0.0
```

To port-forward the User Interface Frontend:

```
kubectl port-forward svc/frontend 3000:80
-n skillpulse --address 0.0.0.0
```

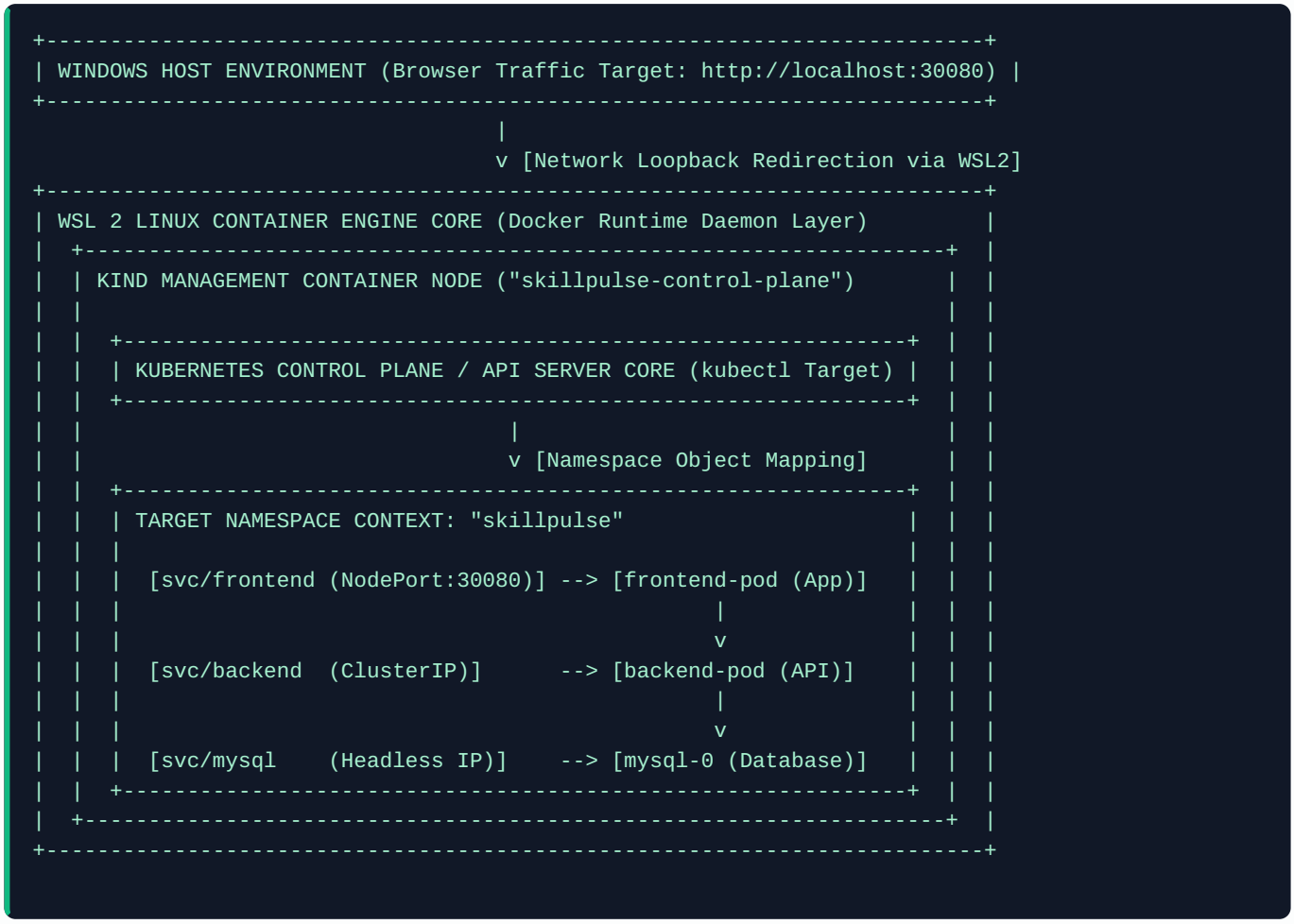
Option B: Permanent NodePort Route

Maps the inner cluster container networks to global loopback sockets securely. Exposes the app permanently on the host machine without dynamic scripts.

Endpoint Target URL:
`http://localhost:30080`

7. Structural Architecture Overview

The communication mesh map below shows how host browser layers, isolated virtualization kernels, and Kind-wrapped cluster node blocks process requests:


WINDOWS HOST ENVIRONMENT (Browser Traffic Target: http://localhost:30080)
[Network Loopback Redirection via WSL2]
WSL 2 LINUX CONTAINER ENGINE CORE (Docker Runtime Daemon Layer)
KIND MANAGEMENT CONTAINER NODE ("skillpulse-control-plane")
KUBERNETES CONTROL PLANE / API SERVER CORE (kubectl Target)
[Namespace Object Mapping]
TARGET NAMESPACE CONTEXT: "skillpulse"
[svc/frontend (NodePort:30080)] --> [frontend-pod (App)]
[svc/backend (ClusterIP)] --> [backend-pod (API)]
[svc/mysql (Headless IP)] --> [mysql-0 (Database)]